Genetic Algorithm vs. Random Search for Configuring Production RAG Retrieval

...

Example retrieval pipeline, Highlighted stages are searched:

By: William Armentrout

The Problem

What RAG does

Retrieves relevant documents at query time and passes them to the model as context [1].

Why it matters

Companies deploy it over internal knowledge bases, where wrong answers carry actual risk.

Who it affects

Enterprise teams and the people relying on the answers. Small teams that can't afford big tuning budgets.

< 15% of 63 reviewed RAG studies addressed real integration into production [2].

What the literature shows

Index choice

Speed vs. recall

IVFFlat builds about 20x faster than HNSW, while HNSW reaches slightly higher reak recall when tuned [3]. IVFFlat can win under low-selectivity filtering [4].

Chunking

Relevance vs. cost

Some chunking strategies improved relevance, but at about 4x the embedding cost [5].

Generalization

Tuned per task

A configuration tuned on one task did not generalize well to a different task,

Takeaway: no single configuration of RAG is best, so each task needs its own tuning, and every trial costs new embeddings and evaluation.

The Gap

Search baseline

Random search

Grid search scales exponentially. Random search is more efficient and is the baseline adaptive methods should try and beat [7].

Evolutionary Methods

GAs for neural nets

GAs have tuned CNN structure and hyperparameters [8] and are increasingly used for architecture searches [9].

RAG tuners

AutoRAG and AutoRAG-HP

A greedy search over RAG modules [10] and a multi-armed bandit formulation [11].

The Gap

GA vs. random search for RAG

None of the sources I found used a GA for RAG configuration or compared it with random search.

Research Question and Hypothesis

RQ: Does a GA find RAG configurations closer to the best achievable than random search at an equal evaluation budget?

Hypothesis:

At an equal evaluation budget, a GA's best configuration will have a smaller gap to the best achievable score than random search's.

Independent var: search method

Dependent variable: gap to best score

Study Design

Type

Controlled computational experiment. Budget, space, questions, and fitness are all held constant.

Data

WixQA enterprise RAG benchmark [12]: questions with labeled relevant passages.

Search space

About 500-1000 configurations.

Genes: chunk size, overlap, retrieval type, reranker toggle, top-k.

Index type fixed.

Grid search

Score every configuration once. This will be used as the ground truth.

Lookup table

Stores nDCG@k for all configurations in the space.

GA (LEAP) and random search

Each run is a table lookup at an equal budget.

Seeded runs

Cheap and repeatable.

This also allows for variance to be reported.

Evaluation and analysis

Fitness

nDCG@k, retrieval only. Relevance is defined once, via overlap with labeled passages.

Hold-out questions

Questions slip into tuning and test sets; configurations are chosen on tuning, scored on test.

Budget, fixed in advance

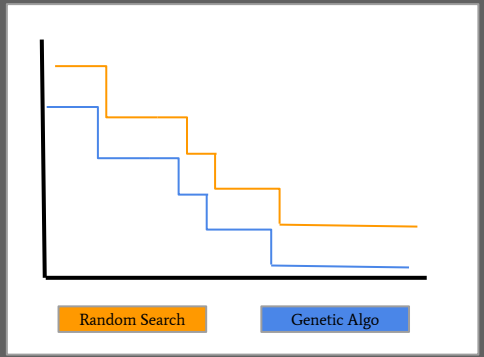
Unique configurations evaluated, revisits counted once. Budgets [5%, 10%, 20%].

Invalid configurations

Handled identically in both methods, egg overlap >= chunksize is resampled.

(sketch not results)

Y: gap to best score. X: unique configurations evaluated.



Analysis: convergence curves, mean, median gap with confidence intervals, Mann-Whitney U test.

Ethical Considerations

Energy and carbon

This study needs the exhaustive grid, the costliest method. Mitigation: run it once and reuse the lookup table.

Dataset Licensing

WixQA has its own terms [12]. Mitigation: follow its rules on use, attribution, and redistribution.

Wrong answers in production

A tuned retriever can still surface the wrong passage, and a high nDCG is not a correct answer. Mitigation: while LLMs are useful it's still recommended for human review.

Overfitting to the questions

A configuration may be best only for the questions it was tuned on. Mitigation: score the chosen configurations on a held-out test set.

Expected contribution and limitations

Expected contribution:

Cost vs. quality. Quantifies how much search cost a GA saves compared with random search, and what it costs in quality.

Reproducible protocol. A pre-scored lookup table plus seeded runs that others can reuse.

Useful even if GA is worse. A null result tells practitioners random search is enough here.

Limitations:

Scope. One dataset (WixQA), retrieval only; answer quality isn't measured.

Size. A small, searchable grid space, so results may not scale well to larger ones.

Cost model. The lookup table hides real per-trial latency and cost.

GA settings. Chosen from prior lab experience, not to be fine tuned.

Summary

Problem : Rag has no universal best configuration, and tuning each task is costly.

Gap: None of the sources I found compared a GA with random search for RAG tuning.

Question : At an equal cost budget, does a GA land closer to the best achievable score?

Method : Lookup table from a full grid, then seeded GA and random-search runs scored on held out questions.

Contribution : Evidence on search cost versus quality.

Thank you

Questions?

References (1 of 2)

[1] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS)*, Vancouver, Canada, 2020.

[2] E. Karakurt and A. Akbulut, “Retrieval-augmented generation (RAG) and large language models (LLMs) for enterprise knowledge management and document automation: A systematic literature review,” *Appl. Sci.*, vol. 16, no. 1, p. 368, 2026, doi: 10.3390/app16010368.

[3] W. Yang, T. Li, G. Fang, and H. Wei, “PASE: PostgreSQL ultra-high-dimensional approximate nearest neighbor search extension,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Portland, OR, USA, 2020, pp. 2241–2253, doi: 10.1145/3318464.3386131.

[4] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu, “Filtered approximate nearest neighbor search in vector databases: System design and performance analysis,” arXiv:2602.11443, 2026.

[5] C. Merola and J. Singh, “Reconstructing context: Evaluating advanced chunking strategies for retrieval-augmented generation,” arXiv:2504.19754, 2025.

[6] M. Barker, A. Bell, E. Thomas, J. Carr, T. Andrews, and U. Bhatt, “Faster, cheaper, better: Multi-objective hyperparameter optimization for LLM and RAG systems,” in *ICLR 2025 Workshop on Foundation Models in the Wild*, 2025, arXiv:2502.18635.

References (2 of 2)

[7] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, 2012.

[8] S. Lee, J. Kim, H. Kang, D.-Y. Kang, and J. Park, “Genetic algorithm based deep learning neural network structure and hyperparameter optimization,” *Appl. Sci.*, vol. 11, no. 2, p. 744, 2021, doi: 10.3390/app11020744.

[9] F. N. Özçelik and M. Ö. Efe, “Evolutionary neural architecture search: A survey,” *Turk. J. Elec. Eng. Comp. Sci.*, vol. 34, no. 4, pp. 507–541, 2026, doi: 10.55730/1300-0632.4189.

[10] D. Kim, B. Kim, D. Han, and M. Eibich, “AutoRAG: Automated framework for optimization of retrieval augmented generation pipeline,” arXiv:2410.20878, 2024.

[11] J. Fu *et al.*, “AutoRAG-HP: Automatic online hyper-parameter tuning for retrieval-augmented generation,” arXiv:2406.19251, 2024.

[12] D. Cohen *et al.*, “WixQA: A multi-dataset benchmark for enterprise retrieval-augmented generation,” arXiv:2505.08643, 2025.